

Capítulo 08

Device

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 31/07/2026

Capítulo 08 — Device (CPU e GPU)

Objetivo de aprendizagem: entender **por que** a GPU acelera o treino de redes neurais — e, mais importante, **quando ela não acelera**. Vamos medir o ponto de virada, o custo de transferir dados, o efeito do tamanho do batch, e escrever código que roda em qualquer dispositivo sem alteração.

Pré-requisitos: Capítulos 1-7. Este capítulo abre a **Fase III** do curso, sobre velocidade e escala.

Arquivos: - `SETUP-GPU.md` — instalação nos três caminhos (NVIDIA / AMD-Intel / sem GPU) - `device.py` — detecção portátil de dispositivo - `benchmark.py` — matmul, transferência e batch: CPU vs GPU - `train_device.py` — treino nos dois dispositivos, em três tamanhos - `exercicios.md` — exercícios

Sobre o hardware desta apostila: os números foram medidos numa **AMD Radeon RX 7600** via **DirectML**, contra uma CPU de desktop. Numa NVIDIA com CUDA os ganhos tendem a ser **maiores** (o CUDA é mais maduro e otimizado). Os *padrões* que os números revelam, porém, valem para qualquer GPU — e é neles que você deve prestar atenção, não nos valores absolutos.

1. Por que uma GPU ajuda

CPU e GPU resolvem problemas diferentes, e a diferença está no desenho:

Característica	CPU	GPU
Núcleos	poucos (4-32), muito rápidos	milhares, individualmente lentos
Otimizada para	latência — terminar <i>uma</i> tarefa rápido	vazão — terminar <i>muitas</i> tarefas por segundo
Cada núcleo	complexo (predição de desvio, cache grande, execução fora de ordem)	simples
Boa em	lógica sequencial, desvios, tarefas variadas	a mesma operação sobre muitos dados

Uma rede neural é, computacionalmente, quase só **multiplicação de matrizes** — e uma matmul é o caso ideal para GPU: milhares de multiplicações independentes, todas iguais, sem desvios. É por isso que a GPU domina deep learning, e não por ser "mais rápida" em algum sentido geral.

A consequência prática — e é o fio condutor deste capítulo:

A GPU só ganha quando há **trabalho suficiente para ocupar os seus milhares de núcleos**. Para trabalho pequeno, o custo fixo de acioná-la é maior que o cálculo, e ela **perde** da CPU.

2. Onde está o ponto de virada

Rodando `python benchmark.py`, multiplicando matrizes quadradas de tamanhos crescentes:

Tamanho	CPU (ms)	GPU (ms)	Speedup	GFLOP/s CPU	GFLOP/s GPU
128	0,02	0,03	0,75x	223	167
256	0,11	0,04	2,90x	296	860
512	0,83	0,10	8,73x	322	2.808
1024	6,25	0,41	15,26x	343	5.240
2048	50,40	3,56	14,14x	341	4.820
4096	393,40	25,02	15,72x	349	5.493

Leia as **duas últimas colunas** — elas contam a história melhor que o speedup:

A CPU satura. Ela entrega ~220-350 GFLOP/s e não passa disso, independentemente do tamanho da matriz. Já está usando tudo o que tem desde matrizes pequenas.

A GPU precisa ser preenchida. Ela começa em 167 GFLOP/s (pior que a CPU!) e sobe até ~5.500 conforme o trabalho cresce — **33 vezes** a própria vazão inicial. Os milhares de núcleos só rendem quando há trabalho para todos.

E em **128×128 a GPU perde** (0,75x). Não é anomalia: é o custo fixo de lançar a operação e coordenar o dispositivo dominando um cálculo minúsculo. Esse é o dado mais importante da tabela, porque contradiz a intuição de que "GPU é sempre mais rápido".

3. Transferir dados custa

Os dados nascem na memória da CPU e precisam ir para a memória da GPU. Isso não é grátis:

Tamanho	MB	CPU->GPU	GPU->CPU	Vazão
256×256	0,3	0,07 ms	0,17 ms	4,0 GB/s
1024×1024	4,2	0,72 ms	1,04 ms	5,8 GB/s
2048×2048	16,8	2,79 ms	3,97 ms	6,0 GB/s

Compare com a Seção 2: transferir uma matriz 1024×1024 custa **0,72 ms**, e multiplicar duas delas na GPU custa **0,41 ms**. Ou seja, **mover o dado custa mais que a conta**. Se você transferir a cada operação, joga fora todo o ganho — e é isso que faz muita gente concluir que "a GPU não ajudou".

Note também que a volta (GPU->CPU) é consistentemente mais lenta que a ida, e que a vazão (~6 GB/s) fica muito abaixo da banda interna da GPU: o gargalo é o barramento PCIe entre os dois mundos, não a memória da placa.

A regra prática que decorre disso:

Mova os dados para a GPU uma vez e deixe-os lá. Todo `.cpu()`, `.item()` ou `print` de um tensor dentro do laço de treino força uma transferência e uma sincronização — a CPU para e espera a GPU terminar. É um dos gargalos mais comuns em código de iniciante.

No `train_device.py` fazemos isso explicitamente:

```
# Move os dados UMA VEZ, fora do laço
Xd, Yd = Xtr.to(device), Ytr.to(device)
```

4. O tamanho do batch importa mais do que parece

Aqui um experimento que isola o efeito: processar **exatamente a mesma quantidade** de vetores, mudando só como eles são entregues.

Batch	Chamadas	Tempo total (ms)
1	4.096	62,80
16	256	4,40
64	64	1,07
256	16	0,39
1.024	4	0,26
4.096	1	0,37

Processar de a **um** custa 62,80 ms; processar tudo de uma vez custa 0,26 ms — uma diferença de **240 vezes** para o mesmo número de multiplicações.

O total de multiplicações é **idêntico** em todas as linhas. A diferença é só o empacotamento — e ela é enorme. Entregar de a um deixa a GPU quase ociosa e paga o custo fixo de lançamento a cada chamada; entregar tudo de uma vez usa a máquina inteira.

É por isso que, ao mudar de CPU para GPU, **aumentar o batch** costuma ser o primeiro ajuste a fazer.

5. Código portátil: nunca crave `.cuda()`

O jeito errado é espalhar `.cuda()` pelo código: aí ele só roda em NVIDIA, e quebra em todo lugar mais. O jeito certo é detectar o dispositivo uma vez e usar a variável:

```
def pegar_device(verbose=True):
    if torch.cuda.is_available(): # 1. NVIDIA
        return torch.device("cuda"), "CUDA"
    try: # 2. AMD/Intel no Windows
        import torch_directml
        if torch_directml.device_count() > 0:
            return torch_directml.device(), "DirectML"
    except ImportError:
        pass
    return torch.device("cpu"), "CPU" # 3. sempre funciona
```

Depois, todo tensor e todo módulo vai para lá:

```
modelo = Modelo().to(dev)
x = x.to(dev)
```

Com isso, os scripts deste capítulo rodam sem alteração em NVIDIA, AMD, Intel ou CPU. Veja [SETUP-GPU.md](#) para a instalação de cada caminho.

6. Como medir GPU sem se enganar

Esta seção existe porque **eu errei a medição na primeira tentativa**, e o erro é tão comum que vale mais que um aviso.

Problema 1: execução assíncrona

Chamadas para a GPU são **assíncronas**. Quando você escreve `c = a @ b`, o Python **não** espera o cálculo: ele enfileira a operação e devolve o controle imediatamente. Se você cronometrar assim:

```
t0 = time.perf_counter()
c = a @ b # SÓ ENFILEIRA
t = time.perf_counter() - t0 # mede microssegundos!
```

...você mede o tempo de *enfileirar*, não de *executar*, e conclui que a GPU é absurdamente rápida. Em CUDA a solução é `torch.cuda.synchronize()`. O DirectML não expõe essa função, então é preciso forçar a fila a esvaziar de outra forma.

Problema 2: sincronizar da forma errada

Minha primeira versão criava **outro** tensor e o copiava para a CPU, esperando que isso esvaziasse a fila. Não é confiável. O resultado foram speedups sem sentido — não monotônicos:

```
512 2.26 3.64 0.62x <- GPU "mais lenta" que em 256
1024 6.42 0.35 18.17x <- e depois "18x mais rápida"
```

Uma curva assim é sinal de medição contaminada, não de comportamento real do hardware. A forma correta é **tocar no próprio resultado**, o que obriga o dispositivo a produzi-lo:

```
def drenar(saida, device):
    if device.type == "cuda":
        torch.cuda.synchronize()
    elif device.type != "cpu" and torch.is_tensor(saida):
        saida.cpu() # lê o RESULTADO -> a fila tem de esvaziar
```

Problema 3: média em vez de mínimo

Mesmo sincronizando certo, sobrava ruído. A correção final foi usar o **mínimo de várias rodadas** em vez da média:

O tempo real de uma operação é um **piso**. Qualquer medição acima dele foi contaminada por outra coisa disputando a máquina — o sistema operacional, outro processo, o próprio Python. A média incorpora esse ruído; **o mínimo o descarta**.

Com as três correções (drenar pelo resultado, aquecer antes, mínimo de 3 rodadas), a tabela da Seção 2 ficou monotônica e coerente. Sem elas, eu teria publicado números errados com aparência de medição.

Problema 4: o primeiro uso é sempre mais lento

A primeira chamada aloca buffers e compila kernels para aquele formato de tensor. Por isso todo benchmark começa com **aquecimento** — algumas execuções descartadas antes de medir.

7. E o nosso modelo, ganha algo?

Agora a pergunta prática. O `train_device.py` treina o Transformer do Capítulo 5 nos dois dispositivos, em três tamanhos, medindo o tempo por passo:

Modelo	Parâmetros	Batch	CPU (ms/passo)	GPU (ms/passo)	Speedup
pequeno (o do Cap. 5)	153.499	64	12,8	42,7	0,30x
médio	3.172.379	256	206,9	78,0	2,65x
grande	18.937.883	512	1.916,8	280,9	6,82x

O nosso modelo é 3,3x MAIS LENTO na GPU. Esse é o resultado mais útil do capítulo, e ele é contra-intuitivo.

O motivo já está na Seção 2. Com `n_embd = 64` e `batch = 64`, as matmuls do nosso modelo são da ordem de 64×64 a 512×64 — exatamente a faixa onde a GPU perde. Somando o custo de lançar

dezenas de kernels minúsculos por passo (mais o `lerp` do AdamW, que cai de volta na CPU — veja [SETUP-GPU.md](#)), o resultado é pior que rodar na CPU.

Conforme o modelo cresce, a conta se inverte: **2,65x** com 3,2 milhões de parâmetros e **6,82x** com 18,9 milhões. E a tendência continuaria: quanto maior o modelo, maior a vantagem — que é exatamente o motivo de LLMs de verdade serem treinados em GPU.

A recomendação honesta para este curso: continue rodando os capítulos na **CPU**. O nosso modelo é pequeno demais para se beneficiar, e a CPU é mais simples de usar. A GPU passa a valer a pena quando você escalar — e agora você sabe medir a partir de quando.

Quanto custa uma operação não suportada

Ao rodar na GPU aparece um aviso: o AdamW usa `aten::lerp`, que o DirectML não implementa, e essa parte **volta a executar na CPU** a cada passo. Quanto isso custa? A solução do exercício E6 mede, no modelo médio:

Otimizador	GPU (ms/passos)	Speedup	Operações em fallback
AdamW	75,2	2,65x	<code>aten::lerp.Scalar_out</code>
SGD + momentum	32,7	5,94x	nenhuma
SGD puro	30,9	6,23x	nenhuma

Trocar o otimizador por um que não usa `lerp` **mais que dobra** o ganho: de 2,65x para 5,94x. Aquela única operação custava ~42 ms por passo — **mais da metade** do tempo total.

Duas conclusões:

1. **Os speedups desta apostila são um piso, não um teto.** Com um backend completo (CUDA) ou um otimizador sem lacunas, a mesma placa daria mais.
2. **Lacunas de backend não aparecem como erro — aparecem como lentidão silenciosa.** O código roda, o resultado está certo, e você simplesmente perde metade da performance sem saber. Rode uma vez com `python -W always::UserWarning` e leia os avisos.

Um efeito colateral: a GPU dá o mesmo resultado?

Modelo	Loss CPU	Loss GPU	Diferença
pequeno	2,3270	2,3270	4,77e-07
médio	2,2308	2,2308	2,38e-07
grande	2,1605	2,1520	8,50e-03

Nos dois primeiros, a diferença é de arredondamento puro ($\sim 1e-07$): a GPU usa kernels diferentes e soma os números em outra **ordem**, e em **float32** a ordem da soma altera o último dígito. É o mesmo efeito que vimos no Capítulo 4, ao comparar as versões da atenção.

Mas no modelo **grande** a diferença salta para **8,5e-03** — quatro ordens de grandeza maior. Isso **não** é um bug, e a explicação importa:

Treinar é um processo **caótico**. Uma diferença de $1e-07$ no gradiente do primeiro passo muda levemente o peso atualizado, o que muda o gradiente do passo seguinte, e assim por diante. Ao longo de 105 passos, esse desvio microscópico é **amplificado**. Quanto maior o modelo, mais operações por passo e mais rápido o desvio cresce.

A consequência prática é relevante e pouco divulgada: **não existe reprodutibilidade bit-exata entre dispositivos diferentes**. Mesmo código, mesma semente, GPUs distintas — trajetórias distintas. Por isso, ao comparar resultados entre máquinas, compare **estatísticas** (loss final, média de várias sementes), nunca valores exatos.

8. Resumo do capítulo

- **CPU otimiza latência; GPU otimiza vazão.** A GPU tem milhares de núcleos simples, e só rende quando há trabalho para todos.
- Medindo matmul: a **CPU satura** em ~ 340 GFLOP/s; a **GPU escala** de 167 até ~ 5.500 . Em matrizes pequenas (128×128) a **GPU perde**.
- **Transferir custa.** Mova os dados uma vez e deixe-os na GPU. `.item()` e `.cpu()` dentro do laço de treino forcem sincronização e viram gargalo.
- **Batch grande é essencial na GPU.** O mesmo trabalho total, entregue em pedaços pequenos, desperdiça a máquina.
- **Escreva código portátil** (`pegar_device()`), nunca `.cuda()` cravado.
- **Medir GPU é sujeito a erro:** execução assíncrona, sincronização mal feita, ruído e aquecimento. Errei os três primeiros na primeira tentativa e documentei a correção.
- **O nosso modelo é 3,3x mais lento na GPU** ($0,30x$), e só passa a ganhar em escala: $2,65x$ com 3,2 M de parâmetros, $6,82x$ com 18,9 M. Para este curso, continue na CPU.
- **Backends imaturos têm lacunas silenciosas:** o AdamW usa uma operação que o DirectML não implementa e que volta para a CPU a cada passo — sem erro, só lentidão. Rode com `-W always::UserWarning` para descobrir.
- **Não há reprodutibilidade bit-exata entre dispositivos.** O treino é caótico: $1e-07$ de diferença no primeiro passo virou $8,5e-03$ depois de 105 passos no modelo grande. Compare estatísticas, não valores exatos.

O que vem no Capítulo 9

Já usamos a GPU. O próximo passo é usá-la **melhor**: no **Capítulo 09 — Precision** vamos treinar com **menos bits por número** (fp16, bf16), o que dobra a vazão e reduz o uso de memória — e entender por

que isso funciona sem estragar o treino, e quando estraga.

-> Antes de seguir, faça os [exercícios](#).

Preparando a GPU — os três caminhos

O código do capítulo (`device.py`) detecta o dispositivo automaticamente, então os scripts **rodam sem alteração** em qualquer um dos casos abaixo. O que muda é a instalação.

Descubra primeiro que placa você tem:

```
# Windows
wmic path win32_VideoController get name
# Linux
lspci | grep -i vga
```

Caminho 1 — NVIDIA (CUDA)

O mais simples e o melhor suportado. Instale a build de CUDA do PyTorch (a versão do `cu###` depende do seu driver — consulte <https://pytorch.org/get-started/locally/>):

```
pip install torch --index-url https://download.pytorch.org/whl/cu124
```

Verifique:

```
python -c "import torch; print(torch.cuda.is_available(),
torch.cuda.get_device_name(0))"
```

Deve imprimir `True` e o nome da placa. Nada mais é necessário — `device.py` já encontra.

Caminho 2 — AMD ou Intel no Windows (DirectML)

Placas AMD e Intel **não** rodam CUDA. No Windows, a alternativa é o **DirectML**, que funciona sobre qualquer GPU compatível com DirectX 12.

Atenção à versão do Python. O `torch-directml` não tem distribuição para todas as versões. No momento em que este capítulo foi escrito, ele exigia **Python 3.10-3.12** (não funciona em 3.13+). Se o seu Python principal for mais novo, crie um ambiente separado só para isso.

```
# Windows PowerShell – use um Python 3.12 (ajuste o caminho)
py -3.12 -m venv C:\dml312
C:\dml312\Scripts\python.exe -m pip install torch-directml
```

Cuidado com caminho longo: o Windows limita caminhos a 260 caracteres, e a instalação do numpy cria caminhos profundos. Se der `WinError 206`, crie o ambiente num diretório curto (`C:\dml312`) em vez de dentro de pastas aninhadas.

Verifique:

```
C:\dml312\Scripts\python.exe -c "import torch_directml;
print(torch_directml.device_count(), torch_directml.device())"
```

Deve imprimir `1 privateuseone:0` (ou mais dispositivos).

Limitações honestas do DirectML:

- É mais lento que CUDA em hardware equivalente e recebe menos otimização.
- **A cobertura de operações é boa, mas não é total** — e as lacunas aparecem em lugares inesperados. Todas as operações do *modelo* funcionam (embedding, softmax, GELU, `masked_fill`, LayerNorm, `cross_entropy`, `multinomial` e o `backward`), mas o **AdamW** usa `aten::lerp`, que **não** é implementada: o PyTorch avisa e executa essa parte na CPU.

UserWarning: The operator 'aten::lerp.Scalar_out' is not currently supported on the DML backend and will fall back to run on the CPU.

O treino continua correto, mas parte do passo do otimizador atravessa a fronteira CPU<->GPU a cada iteração — o que come parte do ganho. Ou seja: **os speedups que medimos neste capítulo são um piso**, não o teto do que a placa poderia dar com um backend completo.

Como detectar isso no seu caso: rode com `python -W always::UserWarning seu_script.py` e procure por "fall back to run on the CPU". Um aviso desses dentro do laço de treino é um gargalo silencioso.

- Não expõe `torch.cuda.synchronize()`; para medir tempo é preciso forçar a sincronização de outra forma (ver a Seção 6 da apostila).

Alternativa no Linux: placas AMD suportam **ROCm**, que é bem mais completo que o DirectML. Se você usa Linux com uma AMD recente, prefira ROCm: <https://pytorch.org/get-started/locally/>

Caminho 3 — sem GPU (CPU)

Nada a instalar: é a instalação padrão do curso. Os scripts detectam a ausência de GPU e rodam na CPU.

Você perde as comparações de velocidade, mas **todo o conteúdo conceitual do capítulo continua acessível** — inclusive porque uma das lições é justamente que, para modelos pequenos, a CPU **ganha**.

Se quiser experimentar GPU sem ter uma, o **Google Colab** oferece GPU gratuita: suba os arquivos do capítulo e rode lá.

Como saber que funcionou

```
python device.py
```

Saída esperada (exemplo, no caminho DirectML):

```
dispositivo: DirectML -> privateuseone:0 (1 disponível)
rotulo: DirectML
torch: 2.4.1+cpu
tensor de teste criado em privateuseone:0: soma = -1.2345
```

O `+cpu` na versão do torch é normal no DirectML: o pacote base é a build de CPU, e o DirectML entra como um backend adicional.

Exercícios — Capítulo 08 (Device)

Faça na ordem. Soluções comentadas em [solucoes/](#) — tente antes de olhar.

Sem GPU? Os exercícios E1, E3 e E7 funcionam na CPU. Os demais precisam de GPU — veja [SETUP-GPU.md](#), ou use o Google Colab (GPU gratuita).

E1 — Latência vs vazão (aquecimento)

Sem rodar código: 1. Por que a GPU **perde** da CPU numa matmul de 128×128? 2. Na tabela da Seção 2, a CPU fica em ~340 GFLOP/s em todos os tamanhos, enquanto a GPU sobe de 167 a 5.500. Explique a diferença em termos de "núcleos ocupados". 3. Uma tarefa que precisa de muitos **if** e desvios roda melhor em CPU ou GPU? Por quê?

E2 — Encontre o SEU ponto de virada

Rode `python benchmark.py` na sua máquina. 1. A partir de qual tamanho de matriz a sua GPU passa a ganhar? 2. Qual o speedup máximo que você observa? Ele estabiliza? 3. Compare os GFLOP/s de pico da sua GPU com a especificação oficial da placa. Você chega perto? (Difícilmente — e a diferença é o que os capítulos 9 e 10 atacam.)

E3 — A armadilha do `.item()`

No laço de treino do `train_device.py`, adicione uma leitura do valor da loss **a cada passo**:

```
historico.append(loss.item()) # força transferência + sincronização
```

1. Quanto o tempo por passo aumenta? Meça com o modelo **médio**, na GPU.
2. Por que essa linha é tão caro? (Duas coisas acontecem: transferência e sincronização.)
3. Como registrar a loss sem pagar esse preço a cada passo? (Dica: a cada quantos passos você realmente precisa dela?)

E4 — Tamanho do batch na GPU

Com o modelo **médio** na GPU, varie o batch: 32, 128, 512, 2048. 1. O tempo por passo cresce proporcionalmente ao batch, ou mais devagar? 2. Calcule o **tempo por exemplo** (tempo do passo ÷ batch). Onde ele é menor? 3. Aumente o batch até dar erro de memória. Qual o limite da sua placa? 4. Cuidado conceitual: batch maior processa mais exemplos por segundo, mas **cada passo dá um passo só** de gradiente. Isso significa que o treino inteiro fica mais rápido? (Releia o E5 do Capítulo 7 sobre a relação entre batch e learning rate.)

E5 — Meça errado de propósito

Na função `cronometrar` do `benchmark.py`, remova a chamada a `drenar()` (tanto a de aquecimento quanto a final). 1. Que speedups a GPU passa a "apresentar"? Eles fazem sentido físico? 2. Por que a medição da CPU **não** é afetada por essa remoção? 3. Esse é um erro que se detecta olhando o quê? (Dica: o que a curva de speedup faz quando a medição está contaminada?)

E6 — Operações que caem de volta na CPU (desafio)

Ao rodar `train_device.py` no DirectML, aparece um aviso: o `AdamW` usa `aten::lerp`, que não é implementada, e essa parte executa na CPU. 1. Rode com `python -W always::UserWarning train_device.py` e liste **todas** as operações que caem de volta. 2. Troque o `torch.optim.AdamW` por `torch.optim.SGD` (que não usa `lerp`). O aviso desaparece? O speedup da GPU **melhora**? 3. Se melhorar: o que isso diz sobre os números da apostila? (Eles são um piso ou um teto do que a placa pode dar?) 4. Em CUDA esse problema não existe. Que lição geral você tira sobre depender de um backend menos maduro?

E7 — Quando vale a pena mudar para GPU? (desafio)

Junte os dados que você mediu. 1. Usando o tempo por passo dos três tamanhos, estime a partir de **quantos parâmetros** (ou de que dimensão `n_embd`) a GPU passa a valer a pena na sua máquina. 2. Considere o custo total: se treinar na GPU exige instalar um ambiente separado e depurar operações não suportadas, a partir de que ponto o ganho justifica o trabalho? 3. Para o modelo **pequeno** deste curso, qual dispositivo você recomendaria? Justifique com números — e note que essa é a resposta contra-intuitiva do capítulo.